Group 10: Trevor Nichols, Pauls Olins, Cristian Sosa
ECSE 318 - Lab 2

Problem 1

The ALU (alu_core, N = 16) is a structural, hierarchical design: arithmetic, logic, shift and compare units sit on one datapath and a 4:1 mux selects the result. The upper op bits pick the unit and the low bits map directly onto that unit's own function field, so no translation logic is needed.

The compare unit reuses the arithmetic unit's subtractor; the critical path runs from op through that subtractor into the compare logic.

Units

- arith_unit – one carry-lookahead adder (lookahead_adder + carry_gen) does every operation. Subtraction is a + ~b + 1 (b inverted, carry-in 1). INC forces b = 0 with +1 on the carry-in; DEC forces b = all ones (−1).
- Overflow – signed overflow is v = (a[15] = b_op[15]) AND (o[15] ≠ a[15]): same-sign addends, opposite-sign sum. Unsigned overflow is the carry out on add and NOT carry (a borrow) on subtract. op[0] selects signed or unsigned. overflow is forced to 0 on every non-arithmetic op.
- logic_unit – sel[1:0] picks AND, OR, XOR or BUF; sel[2] inverts the result, so NOT is the inverted BUF.
- shift_unit – barrel shifter of log2(16) = 4 mux stages; stage k shifts by 2^k with zero fill, sign fill (SRA) or wraparound (rotate). Left shifts reverse the operand, shift right and reverse back. The shift amount is B[3:0].
- comp_unit – has no adder of its own: during a compare the arithmetic unit is forced to SUB. EQ = (A − B = 0), LT = sign(A − B) XOR overflow, GT = NOT(LT OR EQ). The low op bits are an OR mask over {GT, EQ, LT} (e.g. SLE = LT OR EQ), giving C = 0…0001 or 0…0000.

Critical path

The critical path is 32 gate levels, from op[5] through the shared subtractor into the compare logic. This is the longest input-to-output path that static analysis (gates.sh, Yosys ltp) finds in the flattened 16-bit netlist.

Verification

Each unit and the full ALU are checked by a self-checking testbench against an independent integer-math reference, for every op encoding. The unit testbenches all pass.

Log:

```
None

# ALL TESTS PASSED (8946176 vectors)
# ** Note: $finish    : tb/alu_core_tb.v(335)
#    Time: 8946176 ns  Iteration: 0  Instance: /alu_core_tb
```

Problem 2

The processor (cpu) is a parameterized structural design with a PC, instruction register (IR), status register (PSR), register file, RAM, a clock generator and the ALU from Problem 1. Every instruction takes two clocks: FETCH, then EXECUTE.

The same ALU does the instruction's operation in EXECUTE and the PC increment in FETCH.

- FETCH – IR <- RAM[PC]; the ALU computes PC + W (W = RAM words per instruction).
- EXECUTE – dst <- dst OP src through the ALU; register, RAM and PSR writes commit on the clock edge that ends EXECUTE. A taken branch loads PC with the source value.
- PSR – {P, N, Z, C, V} = bits 4…0: parity, negative, zero, carry, overflow, updated by every ALU op with a register destination.
- Register file – 2^R registers, one synchronous write port and two asynchronous read ports.
- RAM – 2^A words of N bits, read asynchronously and written on the clock edge; a load or store can access an aligned group of 1, 2 or 4 words.

Test program and results

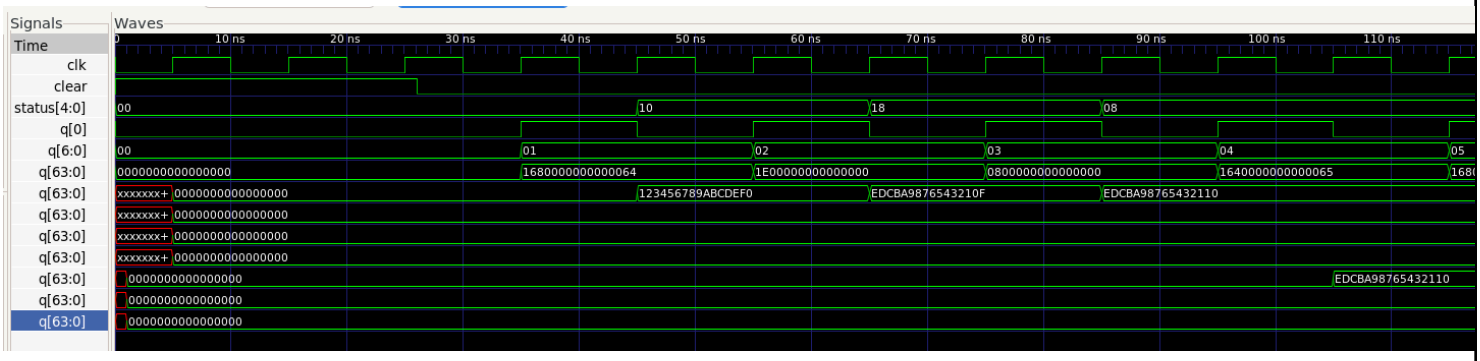
The three tasks run as one program (asm/math.asm) on a 64-bit instance (math_tb), and all three results are correct. asm/asm.py assembles the program into a RAM image that the testbench loads before releasing clear.

- Two's complement – load, not, inc, store: −N = ~N + 1.
- Count of 1s – SWAR mask ladder: add neighbouring bit pairs, then nibbles, bytes, 16- and 32-bit lanes, using the masks 0x5555…, 0x3333… and 0x0F0F… stored in RAM.
- Multiply – shift and add: if the low bit of B is 1, add A to the result; shift A left and B right; loop until B = 0.

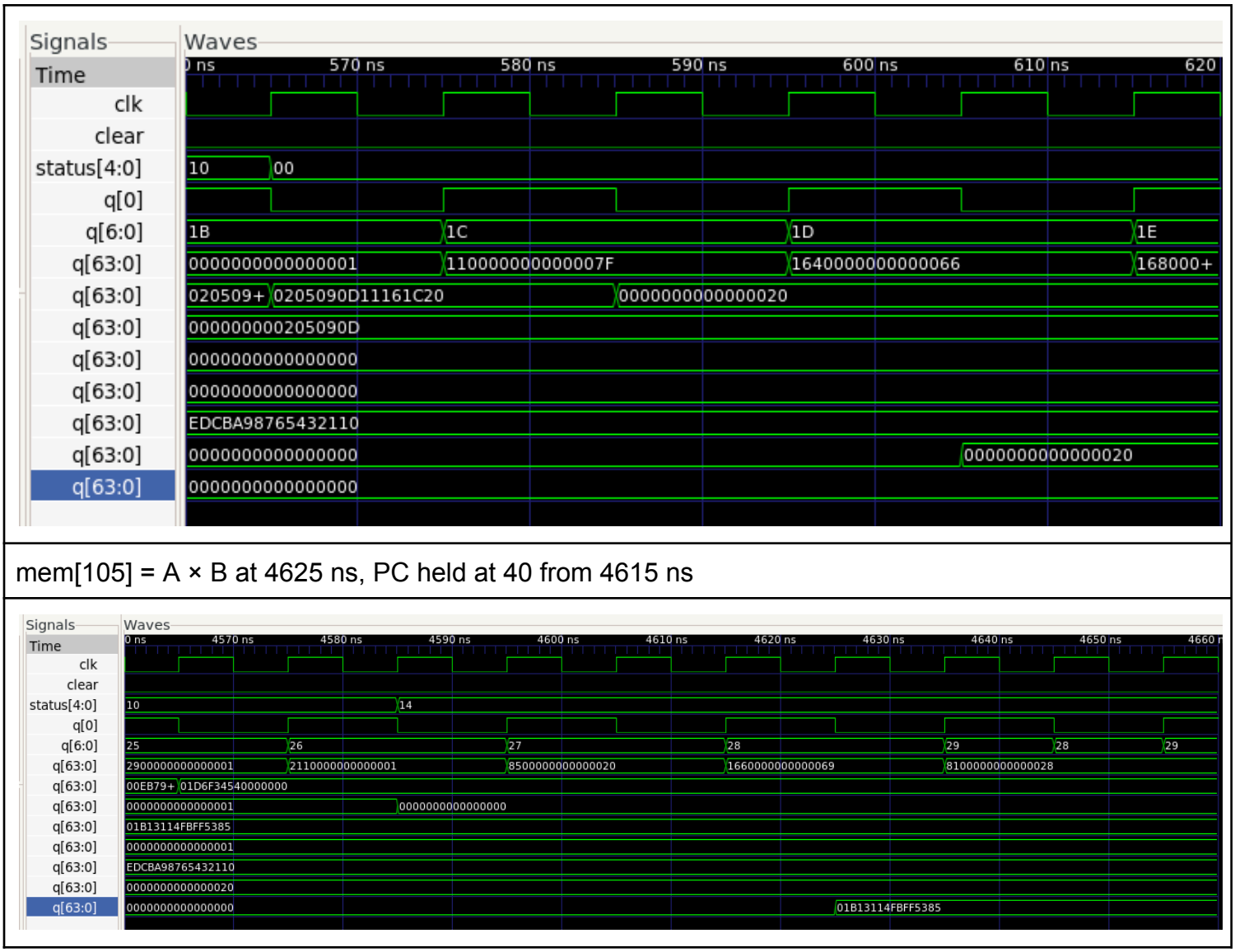
Task	**Input**	**Result location**	**Result**	**Written at**
−N	N = 0x123456789ABCDEF0 (mem 100)	mem 101	0xEDCBA98765432110	105 ns
Count of 1s	same N	mem 102	32	605 ns
A × B	123456789 × 987654321 (mem 103, 104)	mem 105	121932631112635269	4625 ns

Waves:

clear until 26 ns, FETCH/EXECUTE alternating, mem[101] = −N at 105 ns



end of the count, mem[102] = 32 at 605 ns



mem[105] = A × B at 4625 ns, PC held at 40 from 4615 ns

Log:

```
None
# ALL TESTS PASSED (4 vectors)
# ** Note: $finish    : tb/math_tb.v(111)
#    Time: 6026 ns  Iteration: 0  Instance: /math_tb
```

Problem 3

Design

The controller is a state machine with 5 states. There is a 3-bit state register and a 3-bit counter that tracks how long it has been in the current state.

State	**Lights**	**Next state / when to change**
S_AG (reset)	Ga, Rb	Count 6 clocks, move to S_AY only when Sb=1
S_AY	Ya, Rb	Count exactly 2 clocks, then move to S_BG
S_BG	Ra, Gb	Count 5 clocks, then if Sb=1 and Sa=0, go back to S_BG, otherwise go to S_BY
S_BY	Ra, Yb	Count exactly 2 clocks, then move to S_AG
S_FL (fault)	Ra and Rb flash (1 clock on, 1 clock off)	Stay here until reset

The 3 bit state code has 3 unused values (5,6,7). They are all sent to S_FL.

The controller is safe because:

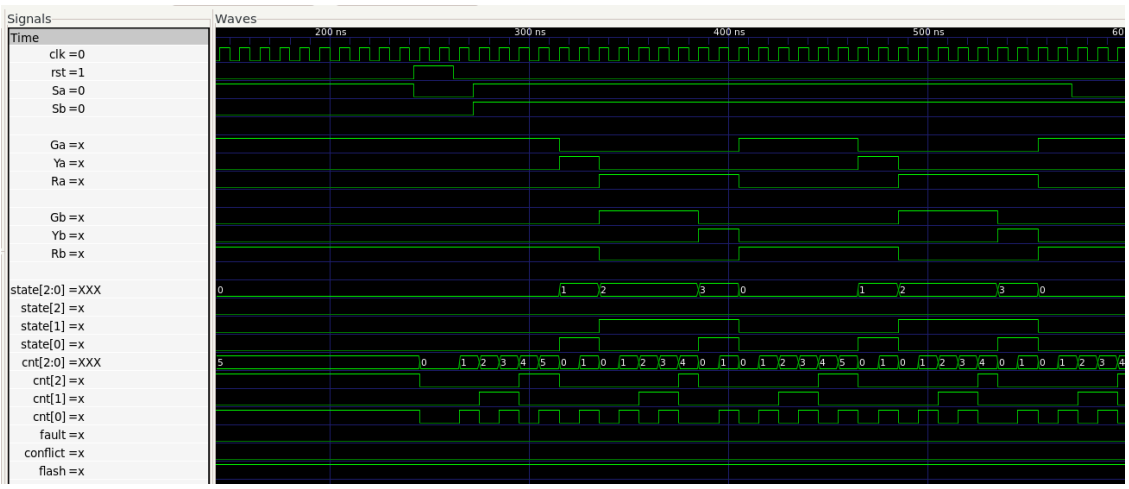
1. Every legal state has a red light on at least one street. The outputs depend only on the state. The table has only 1 non-red light each state, so 2 greens at once cannot happen in normal operation.
2. When going from red to green, the controller always passes through a yellow state. The only way out of a red state is to go through a yellow state, there is no direct transition from a red state to a green state.
3. The controller always starts in a known state. Reset forces S_AG.
4. Corrupted state fails to S_FL. The default for an unused state code goes straight to S_FL.
5. A guard catches any conflicts, so if 2 greens or 2 yellows are even on at the same time, the state is forced to S_FL.

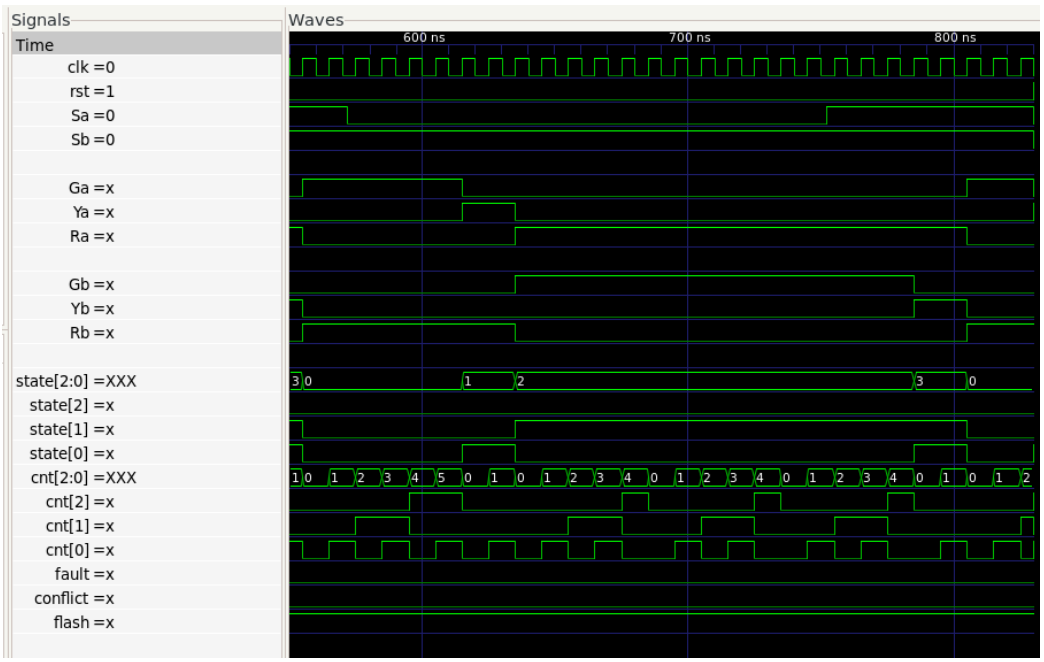
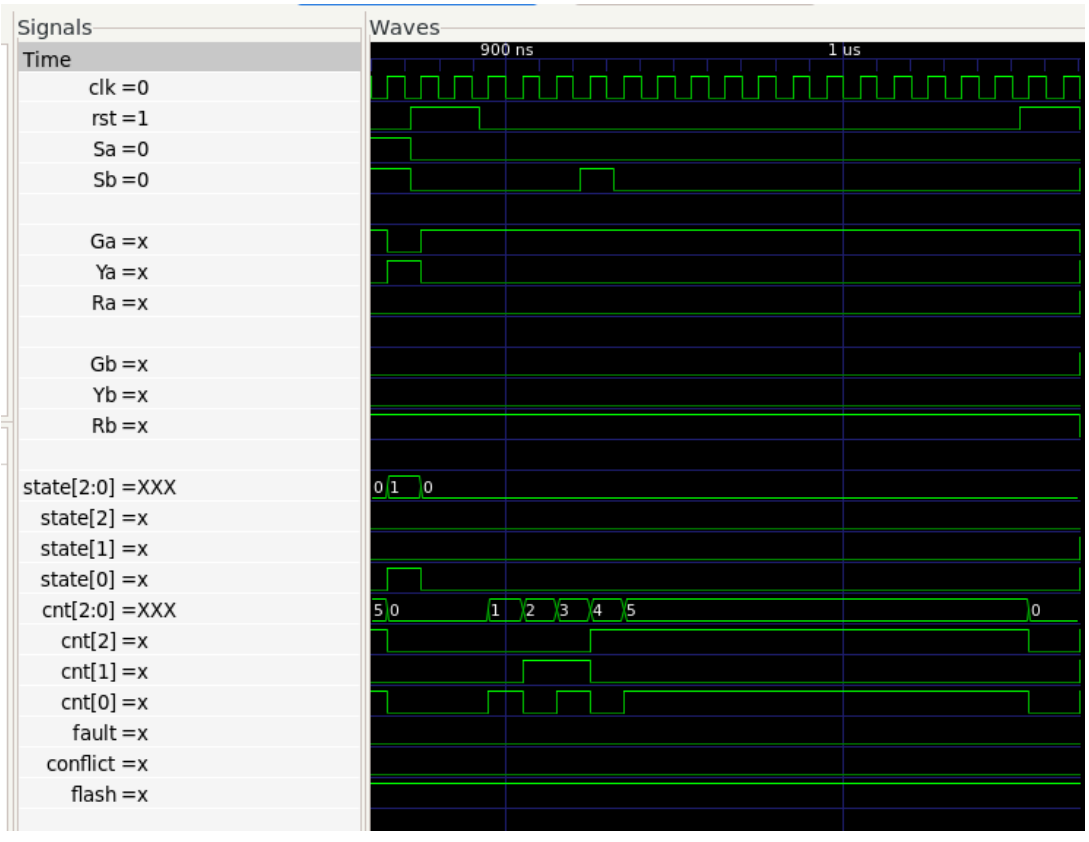
- Log:

None

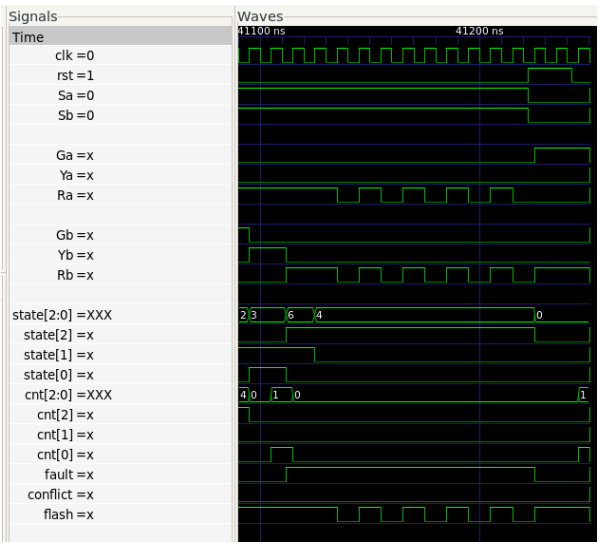
```
# ==================== SUMMARY ====================
#  cycles simulated            : 4142
#  A->B changes                : 199
#  shortest A green seen       : 6 clocks
#  Sb=1 ignored (A green < 6)  : 453 times
#  B green 5-clock extensions  : 86
#  longest B green seen        : 45 clocks
#  RESULT: PASS - 0 errors, no safety violations
# =================================================
# ** Note: $finish    : tb/tb_traffic_controller.v(224)
```

- Waves:
- Normal operation:

Reset, the 6-clock minimum for A green, and the normal cycle	

B green extension	
A stays green when no car is waiting on B	

- Faults:

Fault with forced illegal state - goes S_FL and flashes red on both outputs, until reset	
Fault where Ga is forced on during Gb. machine immediately goes to S_FL and flashes both red	